

Sistema de Gestion de Biblioteca Universitaria

Plan de Implementacion por Fases

Version Web (SGBU)

Proyecto: Biblioteca_Universitaria_Basica

Stack: Next.js 15 + Spring Boot 3.3 + Supabase

Documento para el equipo de desarrollo

Fecha de inicio del plan: 19 de junio de 2026

Ultima actualizacion: 19 de junio de 2026 - Fase 1 completada

1. Proposito del documento

Este documento define el plan de trabajo por fases para implementar el Sistema de Gestion de Biblioteca Universitaria (SGBU) en su version web. Sirve como guía para todo el equipo: establece el orden de avance, los entregables de cada fase, las responsabilidades y las reglas de colaboracion con Git.

2. Estado actual del proyecto (Jun 2026)

Capa	Estado	Observacion
Infraestructura	Completo	Repo, CI, Docker, variables de entorno
Conexion Supabase	Completo	Frontend y backend conectados a PostgreSQL
Fase 1 - Auth y usuarios	Completo	Login, roles, JWT JWKS, logout, guards
Migracion usuarios	Completo	001_create_usuarios.sql aplicada + seed
Modulos de negocio	Pendiente	Libros, prestamos, multas (Fase 2 en adelante)
Integracion FE-BE	Completo	apiFetch envia Bearer token automaticamente

3. Arquitectura acordada

Stack tecnologico

- Frontend: Next.js 15, React 19, TypeScript, Tailwind, Shadcn/UI, TanStack Query
- Backend: Java 21, Spring Boot 3.3, Spring Security, JPA, OpenAPI/Swagger
- Base de datos: Supabase PostgreSQL 15, Supabase Auth, Storage, Realtime

Principios de arquitectura

- Supabase Auth autentica usuarios (login/registro).
- Spring Boot concentra la logica de negocio y acceso a datos via JPA.
- El frontend consume la API REST del backend (no PostgREST directo para negocio).
- Las migraciones SQL viven en supabase/migrations/ y se aplican al proyecto remoto.
- Avance por rebanadas verticales: BD + backend + frontend por modulo.

4. Roles de usuario del sistema

Rol	Descripcion	Permisos principales
ESTUDIANTE	Usuario universitario	Catalogo, sus prestamos, multas y reservas
BIBLIOTECARIO	Personal de biblioteca	CRUD libros/ejemplares, prestamos, devoluciones

ADMIN	Administrador del sistema	Usuarios, reportes, configuracion general
-------	---------------------------	---

5. Modelo de datos (resumen)

El dominio se organiza en torno al catalogo (libros y ejemplares fisicos), las operaciones (prestamos, reservas, multas) y la trazabilidad (usuarios, auditoria).

Relacion principal:

- auth.users (Supabase) -> public.usuarios (perfil + rol)
- libro -> muchos ejemplares (copias fisicas con estado)
- prestamo: usuario + ejemplar + fechas + estado
- multa: derivada de prestamo vencido
- reserva: usuario + libro (cuando no hay ejemplares disponibles)
- log_auditoria: registro transversal de cambios

Estados de ejemplar: DISPONIBLE | PRESTADO | RESERVADO | BAJA

Estados de prestamo: ACTIVO | DEVUELTO | VENCIDO

6. Metodologia de trabajo en equipo

Flujo Git (obligatorio)

- No trabajar directamente en main para nuevas funcionalidades.
- Crear branch: feature/nombre-modulo (ej. feature/usuarios-auth).
- Un branch por modulo o tarea acotada.
- Al terminar: commit claro, push y Pull Request hacia main.
- Revisar PR de un companero antes de merge cuando sea posible.

Orden de trabajo por modulo (cada fase)

Siempre seguir esta secuencia dentro de cada fase:

1. Escribir migracion SQL en supabase/migrations/
2. Aplicar en Supabase (SQL Editor o supabase db push)
3. Backend: entity -> repository -> service -> controller -> DTOs
4. Probar en Swagger (<http://localhost:8080/swagger-ui.html>)
5. Frontend: types, hook TanStack Query, formulario, pagina
6. Prueba manual end-to-end y PR

Division sugerida del equipo (2-3 personas)

Rol en sprint	Responsabilidad
Dev Backend + BD	Migraciones, entidades JPA, API REST, reglas de negocio

Dev Frontend	Pantallas, formularios, hooks, integracion con API
Ambos / QA	Pruebas manuales, revision de PR, documentacion Swagger

Regla de oro: no iniciar una fase nueva hasta cerrar la anterior (merge a main + prueba exitosa).

7. Fases de implementacion

Fase 0 y Fase 1 completadas. El trabajo activo continua en la Fase 2 (Catalogo: Libros).

Fase 0 - Esqueleto del proyecto [COMPLETADA]

Objetivo:

Objetivo: Proyecto compilable y ejecutable con estructura profesional.

Tareas:

- Estructura monorepo: frontend, backend, supabase, docker, CI
- Login basico con Supabase Auth
- Backend con health check, Swagger, security skeleton
- Migraciones placeholder y conexion a Supabase configurada

Entregable:

Repositorio base funcional. Health UP. Login carga correctamente.

Fase 1 - Usuarios y autenticacion real [COMPLETADA]

Objetivo:

Objetivo: Login util con roles; backend reconoce al usuario autenticado.

Tareas:

- Migracion 001: tabla public.usuarios + ENUM rol_usuario + RLS
- Trigger PostgreSQL: sync automatico auth.users -> public.usuarios
- JwtService: validacion JWT via JWKS (ES256) de Supabase
- Endpoints: GET /api/v1/auth/me y GET /api/v1/auth/admin/ping
- apiFetch: Authorization Bearer automatico + manejo 401/403
- Frontend: header con nombre/email/rol, logout, guards por rol
- Rutas por rol: admin->/admin, bibliotecario->/biblioteca, estudiante->/catalogo
- Seed: 4 usuarios de prueba (ver seccion 13)

Entregable:

Login end-to-end verificado. Backend identifica id, email y rol del usuario.

Fase 2 - Catalogo: Libros [PRIORIDAD INMEDIATA]

Objetivo:

Objetivo: El bibliotecario gestiona el catalogo bibliografico.

Tareas:

- Migracion 002: autores, categorias, editoriales, libros
- API REST: CRUD /api/v1/libros con paginacion y busqueda
- Validacion: Bean Validation (backend) + Zod (frontend)
- Pantalla /libros: tabla, filtros, modal crear/editar
- Permisos: solo BIBLIOTECARIO y ADMIN pueden crear/editar

Entregable:

Listar, crear, editar y buscar libros desde la aplicación web.

Fase 3 - Ejemplares

Objetivo:

Objetivo: Gestionar copias físicas del inventario.

Tareas:

- Migración 003: tabla ejemplares (libro_id, código, estado, ubicación)
- Regla: un ejemplar solo puede tener un estado a la vez
- API: CRUD ejemplares, listado por libro
- Frontend: pestaña Ejemplares en detalle de libro

Entregable:

Ver cuantas copias existen y cuales están disponibles por título.

Fase 4 - Prestamos (núcleo del sistema)

Objetivo:

Objetivo: Flujo principal de la biblioteca universitaria.

Tareas:

- Migración 004: prestamos (usuario, ejemplar, fechas, estado)
- Reglas: no prestar ejemplar no disponible; límite prestamos por estudiante
- API: POST /prestamos, PATCH /prestamos/{id}/devolver
- Actualizar estado del ejemplar al prestar/devolver
- Pantalla /prestamos: activos, historial, formulario nuevo préstamo

Entregable:

Prestar y devolver un libro de punta a punta (MVP funcional al cerrar esta fase).

Fase 5 - Multas

Objetivo:

Objetivo: Gestión de cobros por devolución tardía.

Tareas:

- Migración 005: multas (préstamo_id, monto, días_atraso, estado)
- Lógica: job batch (@Scheduled) o proceso al marcar préstamo vencido
- API: consultar multas, marcar como pagada
- Pantalla /multas: estudiante ve las suyas; bibliotecario gestiona

Entregable:

Multas generadas automáticamente y visibles/gestionables en la app.

Fase 6 - Reservas

Objetivo:

Objetivo: Cola de espera cuando no hay ejemplares disponibles.

Tareas:

- Migración 006: reservas (usuario, libro, fecha, estado, notificado)

- Regla: notificar cuando un ejemplar vuelve a DISPONIBLE
- API: crear/cancelar reserva, listar por usuario
- Pantalla /reservas o seccion en libros

Entregable:

Estudiante puede reservar un titulo sin copias disponibles.

Fase 7 - Reportes y administracion**Objetivo:**

Objetivo: Vision gerencial y panel administrativo.

Tareas:

- Endpoints de reportes: libros mas prestados, prestamos activos, multas pendientes
- Pantalla /reportes con graficos o tablas resumen
- Pantalla /admin: gestion de usuarios y roles
- Exportacion basica (CSV/PDF) opcional

Entregable:

Administrador y bibliotecario tienen dashboards de gestion.

Fase 8 - Auditoria, calidad y cierre**Objetivo:**

Objetivo: Trazabilidad, pruebas y preparacion para produccion.

Tareas:

- Migracion 007: log_auditoria + triggers en tablas criticas
- Tests de integracion backend (Spring Boot Test)
- Tests E2E basicos frontend (opcional)
- Documentacion Swagger completa
- Revision de seguridad (RLS, roles, CORS, secretos)

Entregable:

Sistema trazable, probado y listo para despliegue.

8. Cronograma sugerido (referencia)

Semana	Fase	Foco principal
Semana 1	Fase 1	COMPLETADA - Usuarios, JWT, logout, roles
Semana 2	Fase 2	CRUD libros + autores/categorias
Semana 3	Fase 3-4	Ejemplares + prestamos (MVP)
Semana 4	Fase 5	Multas automaticas
Semana 5	Fase 6	Reservas
Semana 6	Fase 7-8	Reportes, admin, auditoria, QA

Nota: el cronograma es orientativo. Ajusten segun disponibilidad del equipo. Lo importante es respetar el orden de fases y no saltar dependencias.

9. Definicion de MVP

El Producto Minimo Viable (MVP) se alcanza al completar las Fases 1 a 4:

- Login con roles (estudiante, bibliotecario, admin)
- Catalogo de libros consultable y editable
- Inventario de ejemplares con estados
- Prestamos y devoluciones funcionales

Con el MVP el sistema ya es demostrable en contexto universitario. Multas, reservas y reportes son incrementos posteriores.

10. Deuda tecnica pendiente

- Migraciones 002-007: aun son placeholders (libros, prestamos, etc.)
- Regenerar database.types.ts automaticamente con Supabase CLI tras cada migracion
- Tests E2E frontend (opcional, Fase 8)
- Documentacion Swagger completa para modulos de negocio

11. Comandos utiles para el equipo

Frontend

```
cd frontend
npm install
npm run dev
-> http://localhost:3000
```


Backend

```
cd backend
.\run-local.ps1
-> http://localhost:8080/api/v1/health
-> http://localhost:8080/swagger-ui.html
```

Git (inicio de tarea)

```
git checkout main
git pull origin main
git checkout -b feature/nombre-modulo
```

12. Criterios de aceptacion por fase

- Migracion aplicada en Supabase sin errores
- Endpoints documentados y probados en Swagger
- Pantalla frontend funcional con datos reales
- PR revisado y mergeado a main
- CI en verde (build frontend + test backend)

13. Fase 1 completada - Resumen tecnico

Que se implemento

- Tabla public.usuarios con roles: admin, bibliotecario, estudiante
- Trigger PostgreSQL para sincronizar auth.users con public.usuarios
- Validacion JWT en backend via JWKS (Supabase ES256)
- Endpoints protegidos: /api/v1/auth/me y /api/v1/auth/admin/ping
- Frontend: login, logout, header con perfil, guards por rol en middleware
- Redireccion automatica: admin->/admin, bibliotecario->/biblioteca, estudiante->/catalogo

Archivos clave

- supabase/migrations/20260618000100_create_usuarios.sql
- supabase/seed/seed.sql
- backend: JwtService, AuthController, Usuario entity
- frontend: middleware.ts, lib/auth/roles.ts, lib/api/client.ts

Como probar la Fase 1

1. Levantar backend: cd backend && .\run-local.ps1
2. Levantar frontend: cd frontend && npm run dev
3. Abrir <http://localhost:3000/login>
4. Iniciar sesion con cualquier usuario de la tabla de la seccion 14
5. Verificar redireccion por rol, header con datos y boton cerrar sesion
6. Probar backend: GET <http://localhost:8080/api/v1/auth/me> (con token Bearer)

14. Usuarios de prueba (SOLO desarrollo)

Usar estas credenciales para probar login y roles. Las contraseñas NO se pueden ver en Supabase (están hasheadas con bcrypt). Documentadas en supabase/seed/USUARIOS-DEV.md.

Rol	Email	Contraseña	Ruta tras login
Admin	admin@sgbu.dev	Admin123!	/admin
Bibliotecario	bibliotecario@sgbu.dev	Biblio123!	/biblioteca
Estudiante 1	estudiante1@sgbu.dev	Est123!	/catalogo
Estudiante 2	estudiante2@sgbu.dev	Est223!	/catalogo

IMPORTANTE: No usar estas credenciales en produccion.

15. Checklist Fase 1

Item	Estado
Login funciona	OK
Logout funciona	OK
JWT validado en backend (JWKS)	OK
Roles y redireccion por rol	OK
Header muestra usuario	OK
Rutas protegidas por rol	OK
Seed cargado en Supabase	OK
Tests backend pasan (mvn test)	OK

Documento generado para el equipo SGBU. Actualizar este plan si cambian requisitos o prioridades del proyecto.